*Classroom Lecture*

# Programming the Raspberry Pi Pico

## with VS Code & MicroPython

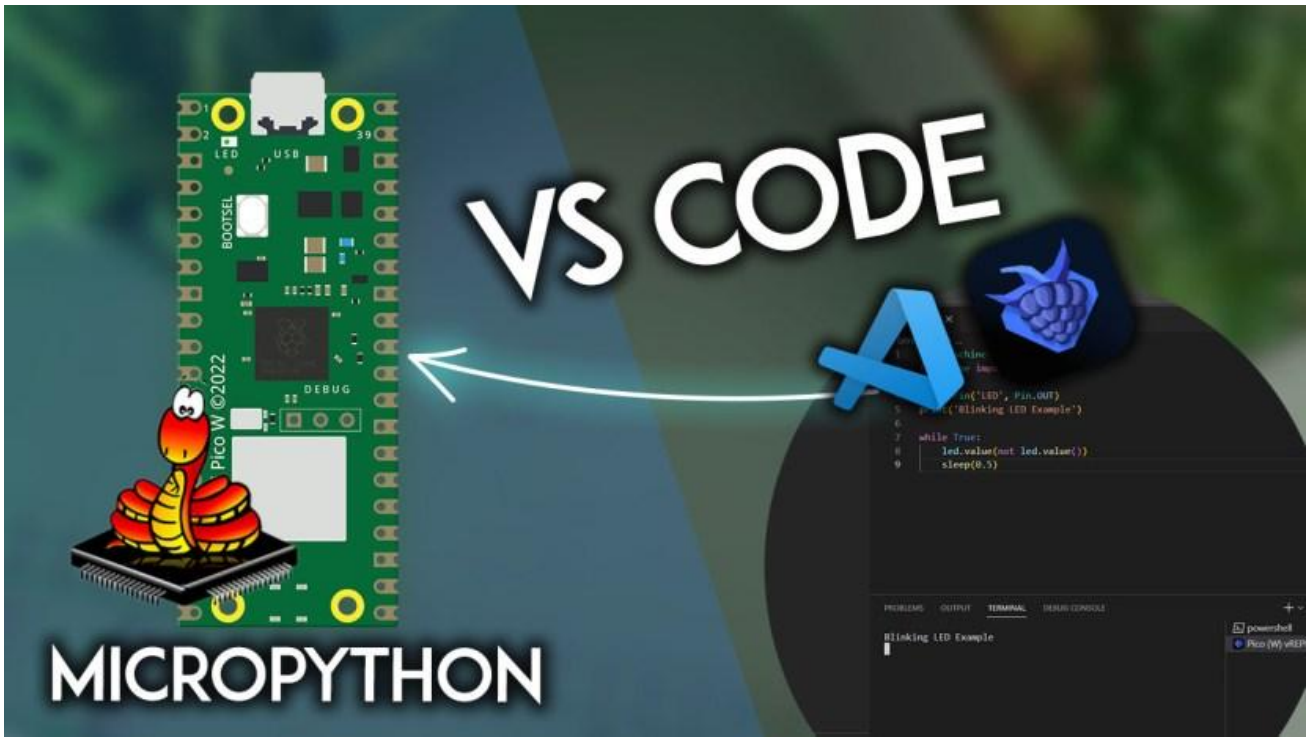
*From first install to running code on real hardware*

| Subject | Level | Duration | Board |
|---|---|---|---|
| RPi Programming and Application | Beginner – Intermediate | 90 – 120 min | Pico / Pico W |

**ℹ Based on / Improved from**

- Original source: Random Nerd Tutorials — 'Programming Raspberry Pi Pico with VS Code and MicroPython'
- This lecture expands the original with instructor notes, concept explanations, deeper code exercises, troubleshooting tables, checkpoints, and extension tasks.

In this guide, you'll learn how to program the Raspberry Pi Pico board with MicroPython using VS Code Editor and the MicroPico extension (previously called Pico-W-Go).



# 1. Lesson Overview & Learning Objectives

By the end of this lesson every student will be able to:

1. Explain what MicroPython is and why it is well-suited for microcontrollers.
2. Install Python, VS Code, and the MicroPico extension on their own machine.
3. Flash MicroPython firmware onto a Raspberry Pi Pico using BOOTSEL mode.
4. Create and configure a MicroPython project folder in VS Code.
5. Write, run, and upload MicroPython programs to the Pico.
6. Read and control digital I/O pins and use the onboard LED.
7. Diagnose and fix the seven most common setup problems.

**Instructor Opening (5 min)**

- Ask: 'Who has programmed a microcontroller before? What language?' — gauge experience.
- Show a Pico and a credit card side by side — point out the chip. Ask what it could be used for.
- Demo: plug in a pre-programmed Pico that blinks an LED. Ask students to predict what the code looks like.

# 2. Background Concepts — What Is MicroPython?

## 2.1 MicroPython in Plain English

Python is one of the most popular programming languages in the world — it runs on laptops, servers, and data-centre clusters. MicroPython is a lean re-implementation of Python 3 designed to run on microcontrollers with as little as 16 KB of RAM.

The Raspberry Pi Pico has 264 KB of RAM and 2 MB of flash storage — far more than MicroPython's minimum — so it is an excellent host. The trade-off is speed: interpreted Python is slower than compiled C, but for most student projects the difference is imperceptible.

**Key Concept — Interpreted vs Compiled**

- Compiled languages (C, C++) translate source code to machine code before running — fast, but requires a build step.
- Interpreted languages (Python, MicroPython) translate and execute line-by-line at runtime — slower but instant feedback via the REPL.
- The REPL (Read-Eval-Print Loop) lets you type Python directly into the Pico and see results immediately — like a calculator for code.

## 2.2 How MicroPython Compares

Use this table to explain to students why we are choosing MicroPython for this class:

| Feature | MicroPython | C / C++ | Arduino IDE |
|---|---|---|---|
| Language | Python 3 subset | C / C++ | C++ dialect |
| Ease of use | Beginner-friendly | Expert-level | Intermediate |
| Speed | Slower (interpreted) | Very fast | Fast (compiled) |
| REPL shell | Yes (interactive) | No | Serial Monitor only |
| Best for | Prototyping, learning | Production firmware | Arduino ecosystem |

## 2.3 The Raspberry Pi Pico — Hardware Overview

The Pico is built around the RP2040 chip, a dual-core ARM Cortex-M0+ processor running at up to 133 MHz, designed in-house by the Raspberry Pi Foundation. Key facts students should know:

- 2 × ARM Cortex-M0+ cores (we will only use one in MicroPython).
- 264 KB SRAM (RAM) — holds running programs and variables.
- 2 MB on-chip Flash — stores MicroPython firmware and your .py files.
- 26 multi-function GPIO pins — digital, PWM, SPI, I2C, UART, ADC.
- 3 × 12-bit ADC channels (GP26, GP27, GP28) and 1 internal temperature sensor.
- Micro-USB port — powers the board AND transfers data / code.
- Onboard LED connected to GPIO 25 (Pico) or controlled via the 'LED' alias (Pico W).

**Instructor Tip**

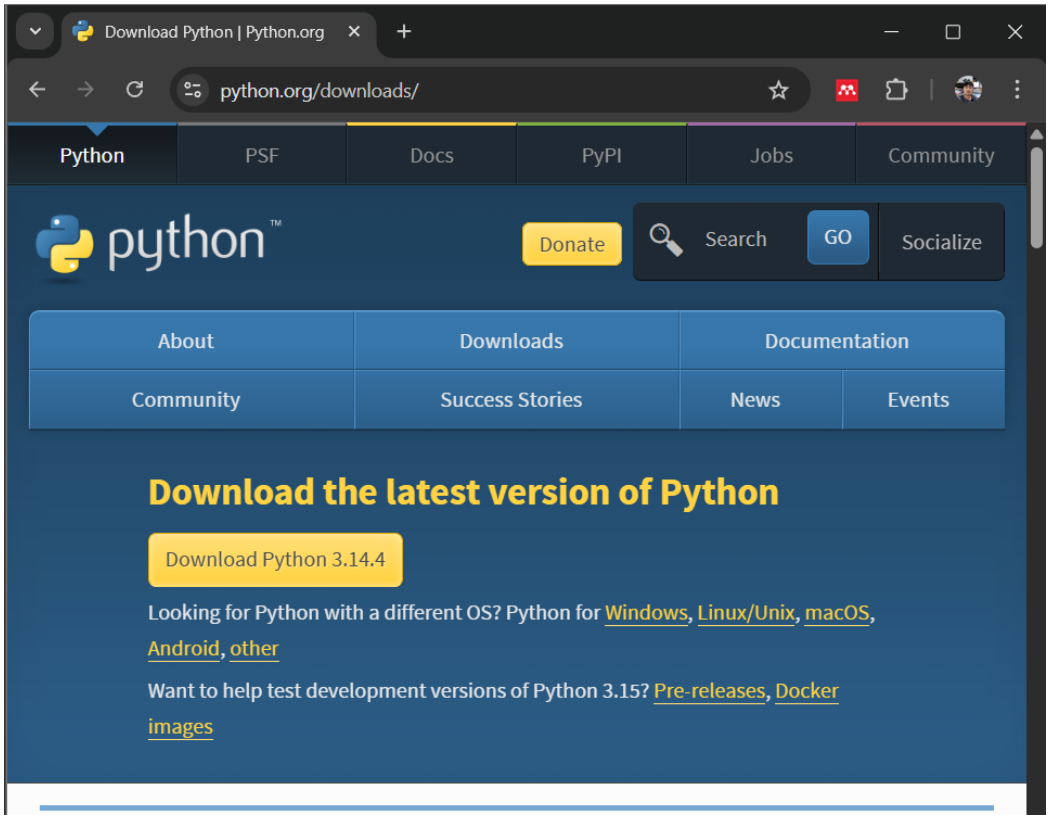
- Pass a real Pico around the class. Point out: the USB connector, the BOOTSEL button, the RP2040 chip, and the 40-pin header.
- Draw a simple diagram on the whiteboard: USB → PC (power + data) → Pico flash. Then: Pico flash → runs main.py at boot.

# 3. Step 1 — Installing Prerequisites

## 3.1 Install Python 3.9 or Newer

MicroPico uses your host computer's Python installation to manage the serial connection. You need Python 3.9 or higher in your PATH.

1. Open a browser and go to python.org/downloads
2. Download the latest stable release for your OS.
3. Run the installer. CRITICAL: on Windows, tick 'Add Python to PATH' before clicking Install Now.




4. After installation, verify it worked. Open a terminal and run:

```
python --version
# Expected output: Python 3.12.x  (or similar 3.9+)
```

5. Install the required pyserial package:

```
pip install pyserial
```

> ⚠ **Windows PATH Trap**
>
> - If 'python' is not recognised after install, you forgot to tick 'Add Python to PATH'.
> - Fix: re-run the installer > Modify > tick 'Add Python to PATH' and continue.
> - Alternative fix: add the Python and Scripts folders manually to System > Environment Variables > Path.

## 3.2 Install Visual Studio Code

VS Code is a free, open-source code editor by Microsoft. It runs on all major operating systems. Follow the column for your OS:

| Windows | macOS | Linux (Ubuntu) |
|---|---|---|
| 1. Visit code.visualstudio.com<br>2. Click Download for Windows<br>3. Run the .exe installer<br>4. Tick "Add to PATH" and "Add Open with Code"<br>5. Complete the wizard and launch VS Code | 1. Visit code.visualstudio.com<br>2. Click Download for macOS<br>3. Open the downloaded .zip<br>4. Drag VS Code to /Applications<br>5. Launch from Applications folder | 1. Download the .deb file<br>2. Open a terminal and run:<br>`sudo apt install ./code_*.deb`<br>3. Find VS Code in app menu |

**ℹ Why VS Code?**

- Autocomplete and IntelliSense — even for MicroPython built-ins, the extension provides type hints.
- Integrated terminal — no need to switch between windows.
- Extension marketplace — thousands of plugins including MicroPico, GitLens, themes, and linters.
- Cross-platform — the same workflow on Windows, macOS, and Linux.

**✓ Checkpoint 1 — Prerequisites Installed**

- python --version prints 3.9 or higher in the terminal.
- pip install pyserial completes without errors.
- VS Code opens and shows the Welcome tab.

# 4. Step 2 — Flash MicroPython Firmware

Before writing any code, MicroPython must be installed on the Pico. This is a one-time process; the firmware stays on the board permanently until you overwrite it.

## 4.1 Download the Correct Firmware

Choose the right .uf2 file for your exact board variant:

| Board | Variant | Download URL |
|---|---|---|
| Raspberry Pi Pico | Standard (no Wi-Fi) | micropython.org/download/rp2-pico |
| Raspberry Pi Pico W | Wi-Fi only | micropython.org/download/rp2-pico-w |
| Raspberry Pi Pico W | Wi-Fi + BLE | raspberrypi.com datasheets page |

## 4.2 Enter BOOTSEL (Bootloader) Mode

The Pico will only accept a new firmware file when it is placed into bootloader mode using the BOOTSEL button.

1. Disconnect the Pico from USB completely.
2. Locate the small white BOOTSEL button on the top of the Pico board.
3. Hold the BOOTSEL button down firmly with one finger.
4. While still holding BOOTSEL, plug the USB cable into your computer.
5. Release BOOTSEL. A new drive called RPI-RP2 should appear in File Explorer / Finder.

> ⚠ **If RPI-RP2 Does Not Appear**
>
> • Most common cause: the USB cable only carries power, not data. Try a different cable.
> • Second cause: BOOTSEL was released too early. Try again — hold until the drive appears.
> • Third cause: USB hub interference. Connect directly to a port on the computer.
> • On Linux: the drive may mount at /media/$USER/RPI-RP2 — check with: ls /media/$USER/

## 4.3 Copy the Firmware

1. Open the RPI-RP2 drive — it contains two files (INDEX.HTM and INFO_UF2.TXT). Ignore these.
2. Drag and drop (or copy-paste) the .uf2 file you downloaded into the RPI-RP2 drive.
3. The Pico will copy the file, reboot automatically, and the drive will disappear. This is normal.
4. The board is now running MicroPython.

> **What Just Happened?**
>
> • The .uf2 format is a special bootloader-aware file format. When copied onto RPI-RP2, the Pico's bootloader writes each block to flash at the correct address.
> • After flashing, the RP2040 boots from flash address 0x10000000 and starts the MicroPython interpreter.
> • The MicroPython REPL is now running on the Pico, waiting for commands over USB serial.

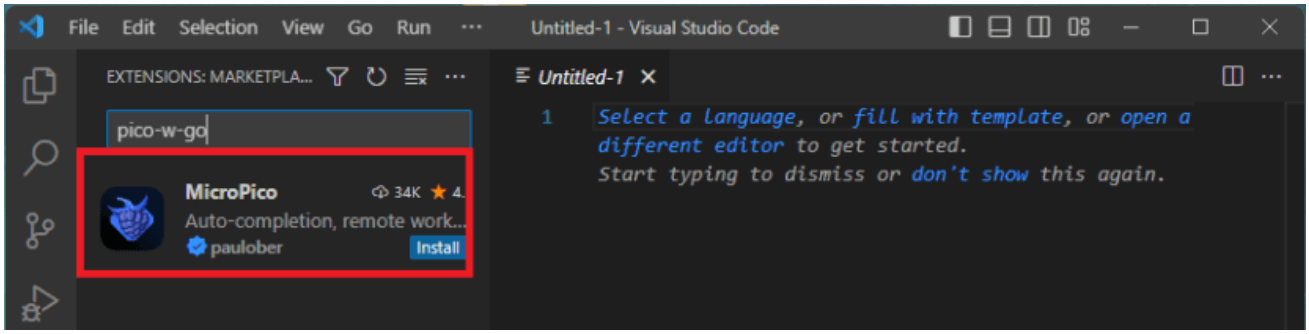
✓ **Checkpoint 2 — Firmware Flashed**

- RPI-RP2 drive appeared after holding BOOTSEL.
- The .uf2 file was copied successfully and the drive disappeared.
- No error messages from your OS during the copy.

# 5. Step 3 — Install and Configure MicroPico Extension

## 5.1 Install the Extension

1. Open VS Code.
2. Click the Extensions icon in the left sidebar (or press Ctrl+Shift+X / Cmd+Shift+X).
3. In the search box, type: MicroPico
4. Find the extension named MicroPico by paulober and click Install.
5. Wait for installation to complete. If prompted, restart VS Code.



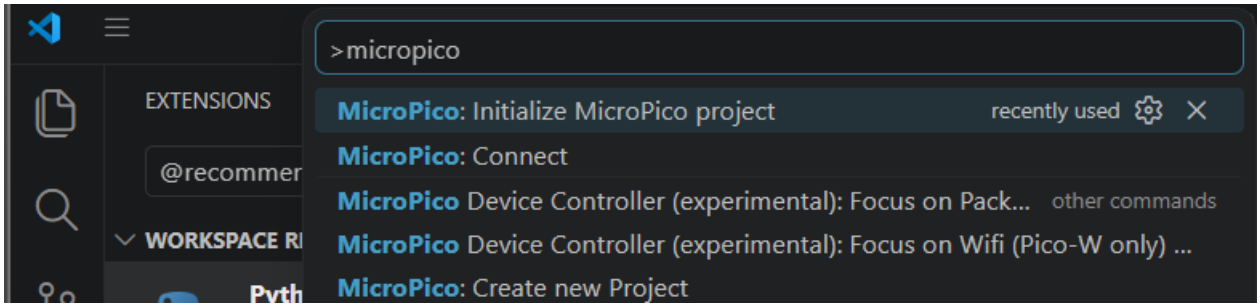

ℹ **About MicroPico**

- **MicroPico** was previously called **Pico-W-Go**. If you see references to the old name in older tutorials, they mean the same thing.
- The extension connects to the Pico over USB serial, gives you a REPL terminal, lets you run files without uploading, and uploads files to the Pico filesystem.
- It also provides IntelliSense stubs for MicroPython modules (machine, utime, etc.) so autocomplete works correctly.

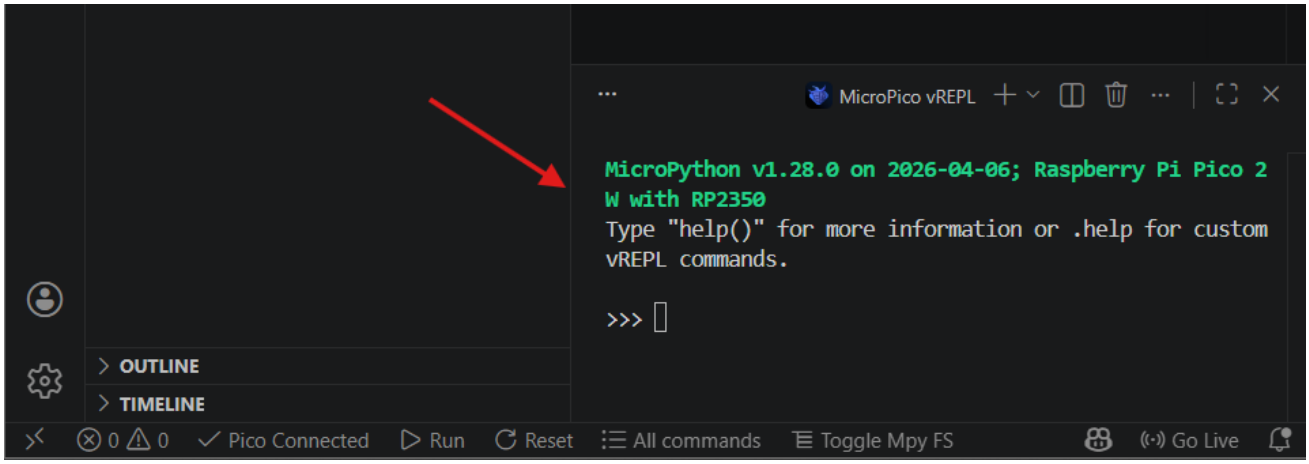
## 5.2 Create and Configure a Project Folder

MicroPico needs a workspace folder to store project configuration files. Follow these steps for every new project:

1. Create a new folder on your PC, e.g. **RPi Pico with VS Code**
2. In VS Code: **File** > **Open Folder** and navigate to the folder you just created.
3. With the Pico plugged in via USB, open the Command Palette: Ctrl+Shift+P (Cmd+Shift+P on Mac).
4. Type micropico and select **MicroPico: Configure project** or **Initialize MicroPico project**.




5. A 'Project configuration complete' message should appear in the bottom-right corner.

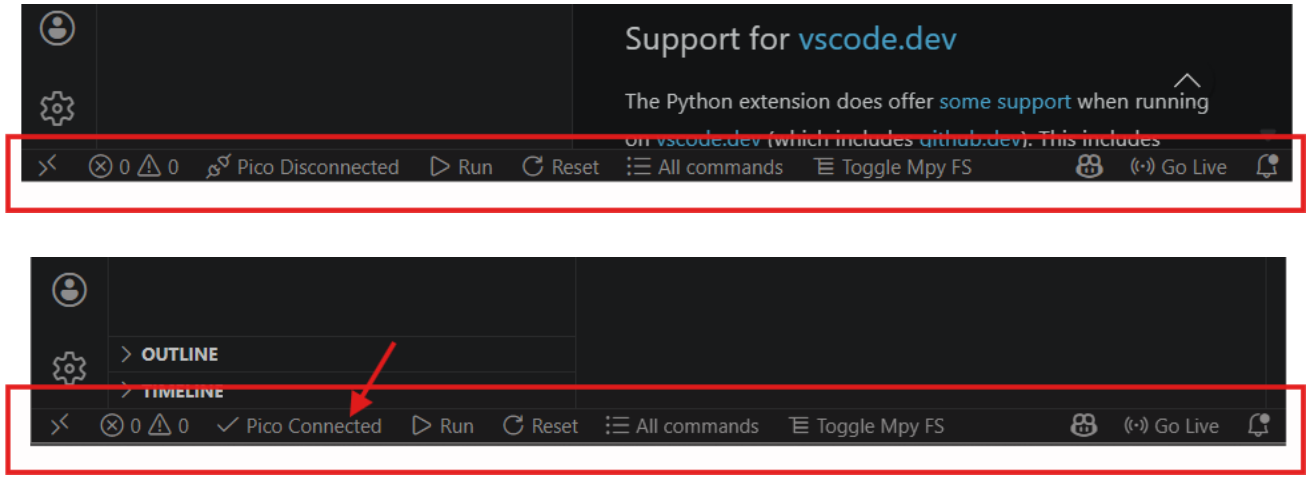



6. VS Code will create these files in your folder:

```
RPi Pico with VS Code/
  .micropico            ← marks folder as a MicroPico project
  .vscode/
    settings.json       ← MicroPython IntelliSense settings
```

## 5.3 The MicroPico Status Bar

After configuration, the bottom bar of VS Code shows MicroPico controls. Familiarise students with each button:



| Status Bar Item | What It Does |
|---|---|
| MicroPico (Connected) | Shows connection status. Click to toggle connection. |
| Run (▶) button | Executes the currently open file on the Pico without saving it permanently. |
| Stop (■) button | Interrupts any running program (sends Ctrl+C to the REPL). |
| Upload button | Copies the current file to the Pico's filesystem permanently. |
| All commands | Opens the full MicroPico command palette for file management and more. |

**⚠ Connected vs Disconnected**

- The status bar must show 'MicroPico Connected' before you can run or upload files.
- If it shows 'Disconnected': ensure the Pico is plugged in, MicroPython is flashed, and no other application (Thonny, PuTTY) has the serial port open.
- • Click the status bar item to manually trigger a reconnect.

**✓ Checkpoint 3 — MicroPico Ready**

- MicroPico extension is installed with no errors in VS Code.
- Project folder has been created and configured (.micropico file is present).
- Status bar shows 'MicroPico Connected'.
- The Run and Upload buttons are visible at the bottom of the window.

# 6. Step 4 — Your First Program: Blinking LED

## 6.1 Write the Code

Inside your project folder, create a new file called blinking.py and type the following code exactly:

```
# blinking.py
# Blink the onboard LED every 0.5 seconds
# Works on both Pico and Pico W

from machine import Pin
from time import sleep

led = Pin('LED', Pin.OUT)    # 'LED' works on both Pico and Pico W
                             # Use Pin(25, Pin.OUT) if you only have original
Pico
print('Blinking LED — press Stop to end')

while True:
    led.value(not led.value())    # toggle: 1 becomes 0, 0 becomes 1
    sleep(0.5)                    # wait 500 milliseconds
```

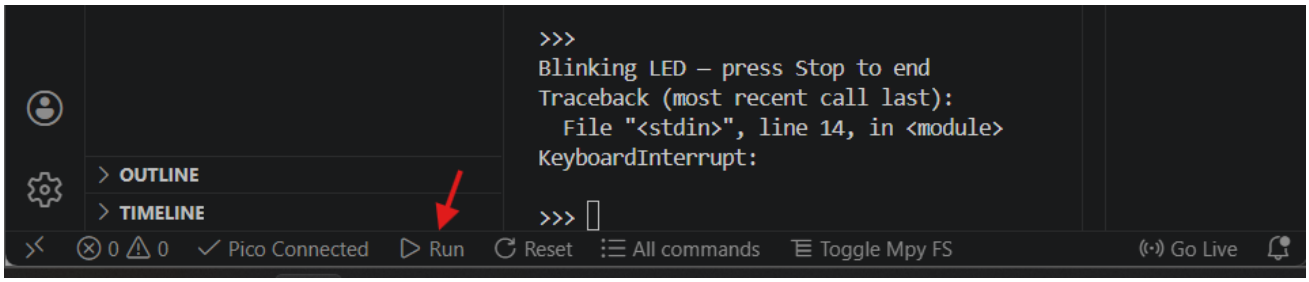
## 6.2 Code Walkthrough

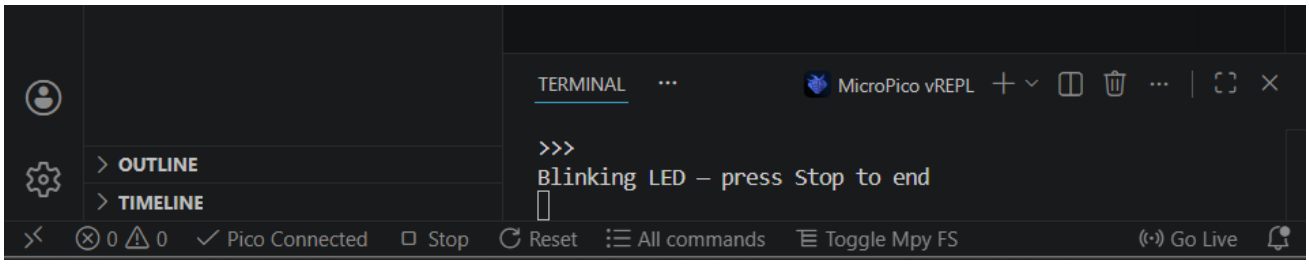
Go through each section with the class before running it:

| Code | Explanation |
|---|---|
| `from machine import Pin` | Imports the Pin class from MicroPython's machine module, which gives low-level access to GPIO pins. |
| `from time import sleep` | Imports the sleep function for creating delays (in seconds). |
| `Pin('LED', Pin.OUT)` | Creates a Pin object. 'LED' is a special alias that resolves to the correct GPIO on both Pico and Pico W. Pin.OUT sets direction to output. |
| `print('...')` | Sends text to the REPL terminal in VS Code so we can confirm the program started. |
| `while True:` | An infinite loop — the indented code inside runs forever until stopped. |
| `led.value(not led.value())` | Reads the current LED state (0 or 1), inverts it (not 0 = 1, not 1 = 0), and writes the new value back — toggling the LED each iteration. |
| `sleep(0.5)` | Pauses for 0.5 seconds. Without this, the LED would toggle millions of times per second, appearing permanently on. |

## 6.3 Run the File (Test Mode)

1. Ensure the Pico is connected and the status bar shows **'MicroPico Connected'**.
2. Click the Run button (▶) at the bottom of VS Code.
3. Watch the onboard LED — it should blink on and off every half second.
4. The REPL terminal should print: Blinking LED — press Stop to end
5. Press the Stop button (■) to halt the program.



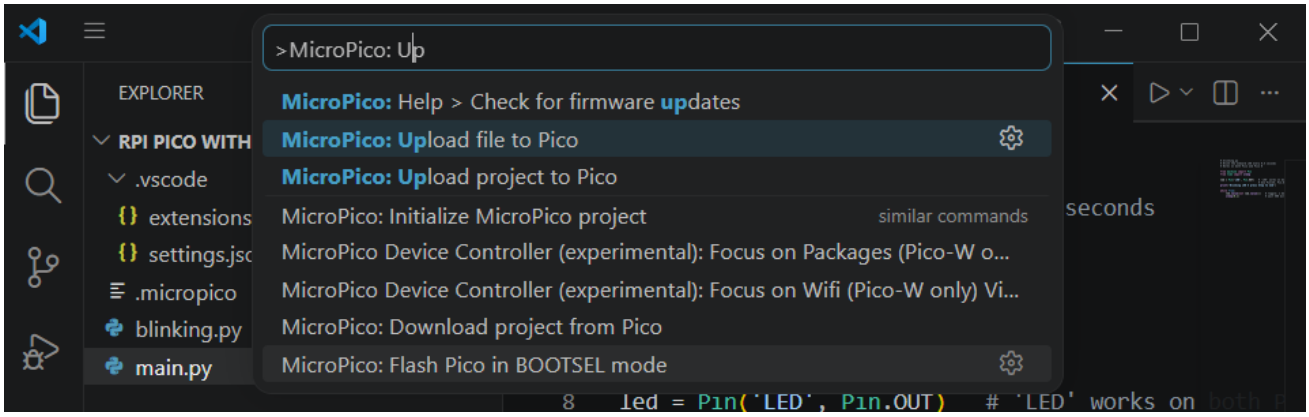




**ℹ Run vs Upload — Important Distinction**

- **Run**: executes the file directly from your PC over USB. It is **NOT saved to the Pico** permanently. If you unplug the Pico, the program is gone.
- **Upload** (main.py): copies the file to the Pico's flash storage. If the file is named main.py, MicroPython runs it automatically every time the Pico is powered on — even without a computer.
- Rule of thumb: use Run for testing, use Upload (named main.py) for deployment.

## 6.4 Upload for Permanent Deployment

1. Create a new file in your project folder called main.py.
2. Copy the exact same blinking code into main.py.
3. Open the Command Palette (Ctrl+Shift+P) and run: MicroPico: Upload current file to Pico.
   or => In VS Code, right-click main.py in the Explorer panel.
   - Select Upload file to Pico.
   - Confirm the upload in the MicroPico status bar.




4. Wait for the success message.
5. Unplug the Pico from USB. Connect it to a USB power bank or USB charger.
6. The LED should start blinking immediately — no computer needed.

> ✓ **Checkpoint 4 — First Program Working**
> - blinking.py runs and the onboard LED blinks at 0.5 s intervals.
> - REPL prints the startup message.
> - main.py uploaded and LED blinks when Pico is powered from a power bank (no PC).

# 7. Step 5 — Going Further: GPIO Input & ADC

## 7.1 GPIO Quick Reference

Students should keep this table open while writing GPIO programs:

| MicroPython Call | What it Does | Example |
|---|---|---|
| `Pin(n, Pin.OUT)` | Configure pin n as digital output | led = Pin(25, Pin.OUT) |
| `Pin(n, Pin.IN, Pin.PULL_UP)` | Input with internal pull-up resistor | btn = Pin(15, Pin.IN, Pin.PULL_UP) |
| `pin.value(1) / pin.value(0)` | Set output HIGH or LOW | led.value(1) |
| `pin.toggle()` | Flip output state | led.toggle() |
| `pin.value()` | Read digital input (0 or 1) | state = btn.value() |
| `ADC(n)` | Analog input on ADC-capable pin | sensor = ADC(26) |
| `sensor.read_u16()` | Read 16-bit ADC value (0-65535) | v = sensor.read_u16() |

## 7.2 Exercise A — Variable Blink Speed with a Button

Wire a push-button between GP15 and GND (the internal pull-up resistor makes external resistors unnecessary). Then upload this program:

```
# button_blink.py
# Normal speed: 0.5 s  |  Button held: 0.1 s (fast blink)

from machine import Pin
from time import sleep

led    = Pin('LED', Pin.OUT)
button = Pin(15, Pin.IN, Pin.PULL_UP)  # PULL_UP: reads 1 when open, 0 when
pressed

while True:
    if button.value() == 0:   # button pressed (pulled LOW)
        delay = 0.1
    else:
        delay = 0.5

    led.toggle()
    sleep(delay)
```

**Instructor Discussion Points**

- Ask: 'Why does the button read 1 when NOT pressed?' — leads to pull-up vs pull-down resistor explanation.
- Ask: 'What would happen if we used Pin.PULL_DOWN instead?' — good thinking exercise.
- Ask: 'If we want 3 blink speeds, how do we change the code?' — introduces elif.

## 7.3 Exercise B — Read the Internal Temperature Sensor

The RP2040 has a built-in temperature sensor on ADC channel 4. This exercise reads it and prints the temperature every second:

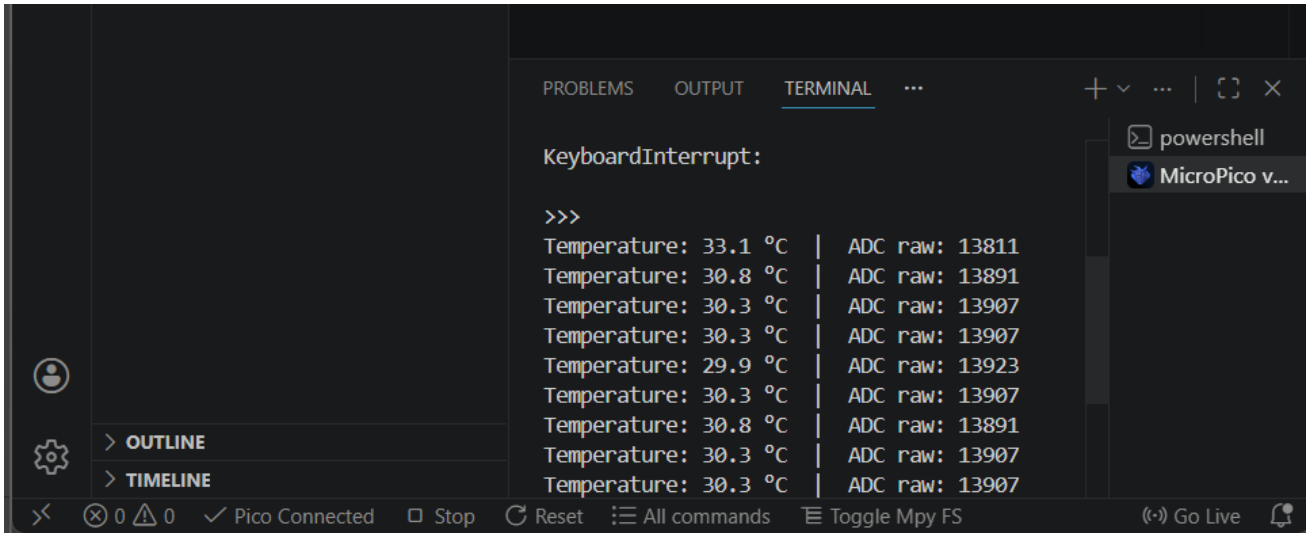
```
# temperature.py
# Read the RP2040 internal temperature sensor

from machine import ADC
import time

sensor = ADC(4)           # ADC channel 4 = internal temperature sensor
VOLT_REF = 3.3            # Pico reference voltage
CONVERSION = 65535        # 16-bit ADC max

while True:
    raw   = sensor.read_u16()                    # 0 - 65535
    volts = raw * (VOLT_REF / CONVERSION)        # convert to volts
    temp  = 27 - (volts - 0.706) / 0.001721     # RP2040 formula
    print(f"Temperature: {temp:.1f} °C  |  ADC raw: {raw}")
    time.sleep(1)
```

Expected output:




### ℹ The Formula Explained

- The RP2040 datasheet gives: T = 27 - (V_ADC - 0.706) / 0.001721
- 0.706 V is the voltage at 27°C; 0.001721 V/°C is the sensor's slope.
- The ADC returns a 16-bit value (0-65535) proportional to 0-3.3 V, so we scale it first.
- The internal sensor is not calibrated for precision — expect ±2-3°C accuracy.

# 8. Troubleshooting — Common Problems & Fixes

Review this table with students before lab time. Most problems students encounter fall into one of these seven categories:

| Symptom | Likely Cause | Fix |
|---|---|---|
| RPI-RP2 drive doesn't appear | BOOTSEL not held before plugging in, or charge-only cable | Hold BOOTSEL before connecting USB. Try a different cable labelled 'data'. Avoid cheap hub. |
| MicroPico shows 'Disconnected' | Pico not plugged in, wrong COM port, or MicroPython not flashed | Check Device Manager (Win) / ls /dev/tty.* (Mac). Re-flash firmware. Restart VS Code. |
| ModuleNotFoundError: machine | Running file with VS Code's built-in Python (not on the Pico) | Always use the MicroPico Run button at the bottom bar, not F5 or the green play button. |
| LED doesn't blink after reboot | File saved as blinking.py, not main.py | MicroPython only auto-runs main.py on boot. Rename and re-upload. |
| Syntax error in REPL on paste | Indentation mixed: spaces and tabs | Set VS Code to 'Spaces: 4'. Use Edit > Convert Indentation to Spaces before pasting. |
| pyserial missing error on connect | pyserial package not installed in host Python | Run: pip install pyserial in your OS terminal, then restart VS Code. |
| Upload hangs or times out | Pico filesystem full | Press Ctrl+C in REPL, then run: import os; os.listdir() to see what's on the board. Delete unneeded files. |

## 8.1 REPL Survival Commands

When something goes wrong, these REPL commands almost always help:

```
# Soft reset the Pico (keeps MicroPython running)
# Press Ctrl+D in the REPL

# Hard stop any running program
# Press Ctrl+C in the REPL

# List all files stored on the Pico
import os
print(os.listdir())

# Delete a file from the Pico
os.remove('old_file.py')

# Check free memory
import gc
gc.collect()
print(gc.mem_free(), 'bytes free')
```

# 9. Extension Challenges

For students who complete the main exercises ahead of schedule. No scaffolding is intentionally provided.

| # | Challenge | Description | Hint |
|---|---|---|---|
| **E1** | **SOS Morse Code** | Make the LED flash SOS (···---···) in an infinite loop, then pause 2 s. | `Each dot = 0.1 s on/off, dash = 0.3 s on/off. Use a list of durations.` |
| **E2** | **Countdown Timer** | Print a 10-to-0 countdown in the REPL with a 1-second pause between each number, then print 'LAUNCH!' | `Use range(10, -1, -1) in a for loop.` |
| **E3** | **LED Dimmer** | Use PWM to fade the LED from off to full brightness smoothly and back, like breathing. | `from machine import PWM. led.duty_u16(value) where value is 0–65535.` |
| **E4** | **Dual-core Hello World** | Run a blinking LED on core 0 AND print a counter on core 1 simultaneously. | `import _thread. _thread.start_new_thread(func, ())` |

# 10. Teacher Notes & Discussion Guide

## 10.1 Lesson Timing

| Time | Activity | Format |
|---|---|---|
| 0 – 10 min | Section 1 & 2: Intro, concept discussion, show Pico hardware | Whole class |
| 10 – 25 min | Section 3: Install Python + VS Code. Students follow on own machines | Individual |
| 25 – 35 min | Section 4: Flash MicroPython firmware. Demonstrate first, then students | Pairs |
| 35 – 50 min | Section 5: Install MicroPico, configure project folder | Individual |
| 50 – 70 min | Section 6: Write, run and upload blinking LED. Walkthrough then students | Individual |
| 70 – 85 min | Section 7: GPIO exercises A and B | Individual / Pairs |
| 85 – 95 min | Section 9: Extension challenges for early finishers | Individual |
| 95 – 120 min | Debrief, show-and-tell, discussion questions (Section 10.2) | Whole class |

## 10.2 Discussion Questions

1. MicroPython is interpreted. In what types of project would the speed difference compared to C matter? When would it not?
2. The Pico's onboard LED is wired to GPIO 25 on the original Pico, but Pico W uses the wireless chip to drive it. How does using Pin('LED') help us write portable code?
3. What is the difference between RAM and Flash on the Pico? What happens to each when you disconnect power?
4. If you wanted the Pico to read a temperature and send it to a web server every minute, what else would you need? (Guides discussion toward Pico W Wi-Fi, HTTP/MQTT, etc.)
5. The internal temperature sensor has ±2-3°C accuracy. In what real-world applications would this be acceptable, and in which would it not?

## 10.3 Differentiation

- Support: Provide a pre-wired breadboard example and printed step-by-step cards for sections 3-6. Pair struggling students with a peer mentor.
- Core: Follow the lecture as written, completing all exercises in sections 6 and 7.
- Extension: Extension challenges in section 9, especially E3 (PWM) and E4 (dual-core threading).
- Advanced: Challenge students to rewrite temperature.py to log readings to a CSV file on the Pico's filesystem using open() and write().

# 11. Quick-Reference Card

Print or display at each workstation.

| VS Code Keyboard Shortcuts | REPL Quick Commands |
|---|---|
| `Ctrl+Shift+P / Cmd+Shift+P` → Command Palette<br>`Ctrl+Shift+X / Cmd+Shift+X` → Extensions panel<br>`Ctrl+D (in REPL)` → Soft reset Pico<br>`Ctrl+C (in REPL)` → Stop running program<br>`Ctrl+`` → Toggle integrated terminal | `import os; os.listdir()` → List files on Pico<br>`os.remove('file.py')` → Delete a file<br>`import gc; gc.mem_free()` → Check free RAM<br>`from machine import Pin` → GPIO access<br>`from machine import ADC` → Analog input<br>`from time import sleep` → Delays |

*— End of Lecture Notes —*